

Revisando o que discutimos sobre programação assistida e automação, entendi que estamos vivendo uma mudança de paradigma real na Engenharia de Software com base no material que a gente viu. Não é que a IA vai fazer o nosso trabalho sozinha, mas ela funciona como um amplificador das nossas capacidades. Essa tendência é confirmada por relatórios globais de instituições como McKinsey, Gartner e o MIT, que mostram como a automação impacta a produtividade e a qualidade de vida profissional. Para mim, ficou claro que ferramentas como o GitHub Copilot e o Cursor são como a calculadora, a qual não substitui o matemático mas faz o "trabalho sujo", as LLMs agilizam o código repetitivo ou boilerplate e até aumentam a velocidade de entrega em uns 35%, mas não são infalíveis.

Eu percebi que o grande desafio para a gente é desenvolver esse "novo letramento". A IA pode sugerir uma solução que funciona, mas que é ineficiente ou insegura. Durante nossas práticas com ferramentas como Copilot, Replit e seu Ghostwriter, praticando sugestões de código e refatoração, vimos que ela pode te entregar um algoritmo com complexidade técnica ruim, ou te entregar códigos viciados, com viés ou muitas falhas de segurança, e até mesmo códigos com licenciamento de outras empresas. Analisando cenários reais, como o da empresa PG&E, vimos que a IA ajudou a economizar mais de 500 mil horas de trabalho manual, mas o risco de dependência e de aceitar códigos vulneráveis sem revisão continua sendo o maior gargalo. Por isso, o workflow sustentável exige que a IA participe não só da implementação, mas também do planejamento, revisão por pares, testes, documentação e manutenção, sempre sob nossa transparência e supervisão.

Somando a esse cenário real, o caso da PG&E demonstra que a eficiência operacional aumentou drasticamente, reduzindo o tempo de processamento de aplicações em 40% e economizando 1.400 horas de pesquisa logo no primeiro mês. Entretanto, esse avanço traz o que chamamos de "Trust Paradox" (Paradoxo da Confiança): o risco de aceitarmos códigos que apenas "parecem" corretos. Dados recentes indicam que o código gerado por IA pode introduzir milhares de novas vulnerabilidades por mês se não houver uma análise rigorosa. A falta de contexto de segurança da IA e a "Ilusão de Momentum" — onde a velocidade de escrita oculta a necessidade de refatoração — podem gerar uma dívida técnica difícil de manter no futuro.

O ponto que mais me fez refletir foi uma pergunta que o podcast levantou: "Dominará o mundo é: Quem domina as ferramentas de IA? Ou quem domina os fundamentos da ciência da computação?"

Isso nos leva ao debate que fizemos em grupo: "Até que ponto podemos confiar na IA como parceira de código?". No fim das contas, o que eu entendi é que a resposta não é um ou outro. A responsabilidade final é sempre nossa. A IA é o copiloto, mas quem precisa saber os fundamentos da linguagem, entender a lógica por trás de cada linha e garantir a segurança do sistema somos nós programadores. É a velha história da calculadora: ela ajuda a fazer a conta rápido, mas quem tem que saber a matemática para interpretar o resultado é o matemático. Essa base sólida em fundamentos é, inclusive, o que vai nos preparar para os desafios da próxima unidade sobre Low Code e No Code.